

Universidade do Minho

PROJETO SO

Trabalho Prático

RUNNER E CONTROLLER

Universidade do Minho
10 de Maio de 2026

Gonçalo Miguel Abreu Pereira a 111016, LCC
João Pedro Castro Alves a110837, LEI
Simão Alves Nogueira a111266, LEI

Índice

- **Resumo**
- **1.Introdução**
- **2. Arquitetura de Processos**
  - 2.1 Visão Geral
  - 2.2 Ciclo de Vida de um Pedido de Execução
  - 2.3 Hierarquia de Processos
- **3. Mecanismos de Comunicação**
  - 3.1 FIFOs (Pipes com nome)
  - 3.2 Protocolo de Mensagens
  - 3.3 Pipes Anónimos (Pipeline de Comandos)
- **4**. **Funcionalidades Implementadas**
  - 4.1 Execução de Comandos (-e)
  - 4.2 Suporte a Redirecionamentos e Pipelines
  - 4.3 Consulta de Estado (-c)
  - 4.4 Shutdown Controlado (-s)
  - 4.5 Registo Persistente (log.txt)
- **5. Política de Escalonamento**
  - 5.1 Descrição — Round-Robin por Utilizador
  - 5.2 Implementação
- **6. Avaliação Experimental**
  - 6.1 Metodologia
  - 6.2 Experimento 1 — Impacto do Paralelismo
  - 6.3 Experimento 2 — Fairness entre Utilizadores
  - 6.4 Experimento 3 — Consulta durante Execução
  - 6.5 Experimento 4 — Redirecionamentos e Pipelines
- **7. Decisões de Design e Limitações**
  - 7.1 O_RDWR no controller_pipe
  - 7.2 Identificador Único por Comando
  - 7.3 Atomicidade das Escritas
  - 7.4 Limitações Conhecidas
- **8. Conclusão**

ÍNDICE TABELAS

Tabela 1: Componentes do sistema e os seus papéis
Tabela 2: Campos da estrutura Msg
Tabela 3: Impacto do grau de paralelismo
Tabela 4: Tempos médios reais por utilizador (Round-Robin)
Tabela 5: Resultados reais dos testes de redirecionamento

Resumo

Este relatório descreve o desenvolvimento de um sistema de gestão de tarefas em ambiente LINUX, composto pelos programas controller e runner. O objetivo principal foi implementar um gestor de processos capaz de controlar o paralelismo de execução e gerir uma fila de espera de forma ordenada.

A solução utiliza Pipes Nominais (FIFOs) como base da comunicação inter-processos (IPC), garantindo a integridade dos dados através de um sistema de canais privados por PID. O sistema destaca-se pela implementação de um escalonador FIFO, pela monitorização precisa de tempos de execução com a função gettimeofday e pelo registo persistente de atividades em ficheiros de log.

Os testes efetuados através de scripts de automação em Bash demonstram que o sistema é robusto sob carga, respeita os limites de concorrência definidos e garante a persistência dos dados de auditoria, cumprindo integralmente os requisitos de escalabilidade e eficiência propostos no enunciado.

1. Introdução

O presente relatório descreve a arquitetura, as decisões de design e os resulta dos experimentais do sistema composto pelos programas runner e controller, desenvolvidos em linguagem C no âmbito da unidade curricular de Sistemas Operativos. O objetivo é permitir a execução concorrente e controlada de comandos Unix por parte de múltiplos utilizadores, com suporte a operadores de shell (pipes e redirecionamentos), controlo de paralelismo configurável e uma política de escalonamento justa entre utilizadores.

O sistema é composto por dois programas independentes que comunicam exclusivamente através de pipes com nome (FIFOs), respeitando os requisitos de não utilizar printf, perror ou system() nas funcionalidades principais

2. Arquitetura de Processos

2.1 Visão Geral

O sistema segue uma arquitetura cliente–servidor assíncrona. O controller é o servidor central que permanece em execução, recebendo pedidos de múltiplos processos runner (clientes) em simultâneo. Cada invocação do runner cria um processo independente que comunica com o controller através de FIFOs.

Componente	**Papel**	**Nº. de processos**
controller	Servidor central de escalonamento	1 (daemon)
runner -e	Submete e executa um comando	1 por invocação
filho do runner	Processo que corre o comando efetivo	1 por estágio de pipeline
runner -c	Consulta o estado do controller	1 por invocação
runner -s	Envia sinal de shutdown	1 por invocação

Tabela 1 — Componentes do sistema e os seus papeis.

2.2 Ciclo de vida de um pedido de execução

Quando o utilizador invoca ./runner -e <user_id> <comando>, o processo runner:

- Preenche uma mensagem Msg com o PID (identificador único), o user_id e o comando.
- Cria um FIFO privado em tmp/fifo_<PID> para receber a resposta do controller.
- Escreve a mensagem no FIFO partilhado tmp/controller_pipe.
- Bloqueia em leitura no seu FIFO privado, aguardando autorização.
- Após receber autorização, lança a pipeline com fork() e execvp().
- Ao terminar, envia uma mensagem de conclusão ao controller e remove o FIFO privado.

2.3 Hierarquia de processos

Para suportar pipelines com múltiplos estágios separados por |, o runner faz fork() uma vez por estágio, ligando-os com pipe(). O processo pai aguarda a conclusão de todos os filhos com wait(). Apenas depois envia a notificação de conclusão ao controller, garantindo que o tempo registado corresponde ao tempo real de execução do comando completo.

3. Mecanismos de Comunicação

3.1 FIFOs (Pipes com nome)

Toda a comunicação entre processos é feita exclusivamente via FIFOs (pipes com nome criados com mkfifo()), sem memória partilhada, sockets ou sinais. Foram utilizados dois tipos de FIFO:

• tmp/controller_pipe — FIFO partilhado, criado pelo controller na inicialização. Todos os runners escrevem pedidos neste FIFO. É aberto em modo O_RDWR pelo controller para evitar que o read() retorne 0 (EOF) quando não há runners ativos.

• tmp/fifo_<PID> — FIFO privado, criado por cada runner antes de enviar o pedido. O controller escreve a resposta neste FIFO (autorização ou dados de consulta). É eliminado pelo runner após ser lido.

3.2 Protocolo de mensagens

A estrutura Msg é a única unidade de comunicação entre runner e controller. É escrita e lida de forma atómica, garantindo que não há entrelaçamento de mensagens de diferentes runners.

Campo	**Tipo**	**Descrição**
runner_pid	pid_t	PID do runner – identificador único do comando
type	int	1=executar,2=consultar, 4=terminou, 5=shutdown
user_id	int	Identificador do utilizador
command	char[300]	Comando a executar (com argumentos)
start_time	struct timeval	Instante de chegada ao controller (para medir duração).

Tabela 2 — Campos da estrutura Msg.

3.3 Pipes anónimos (pipeline de comandos)

Para suportar o operador |, o runner usa pipe() para criar canais anónimos entre estágios consecutivos da pipeline. O stdout de cada processo filho é ligado ao stdin do seguinte via dup2(). Os redirecionamentos >, 2> e < são aplicados com open() + dup2() no contexto do filho, antes do execvp(), sem recurso ao interpretador de comandos.

4. Funcionalidades Implementadas

4.1 Execução de comandos (-e)

A opção -e <user_id> <comando> permite ao utilizador submeter um comando para execução. O runner notifica o utilizador de três eventos via stdout: submissão, início de execução e conclusão. O comando só é executado após autorização explícita do controller, garantindo o respeito pelo limite de paralelismo.

4.2 Suporte a redirecionamentos e pipelines

O parser interno do runner identifica os operadores |, >, 2> e < diretamente na string do comando, sem invocar qualquer shell. O processo de parsing divide o comando em estágios (Cmd) e, para cada um, aplica os redirecionamentos antes do execvp().

Exemplos suportados:

- ls bin | wc -l > resultado.txt
- wc -l < input.txt > out.txt
- cat ficheiro.txt | sort | uniq > ordenado.txt
- ls /inexistente 2> erros.txt

4.3 Consulta de estado (-c)

A opção -c envia um pedido de consulta ao controller (tipo 2) e aguarda a resposta no FIFO privado. O controller responde com o estado completo: número de slots em uso, número de comandos em fila, e para cada comando o PID, o user_id e o texto do comando. A consulta não bloqueia o processamento de outros pedidos, pois o controller processa todas as mensagens de forma sequencial.

4.4 Shutdown controlado (-s)

A opção -s envia uma mensagem de tipo 5 ao controller. O controller regista o pedido mas só termina efetivamente quando não existirem comandos em execução nem em fila de espera, garantindo que nenhum trabalho é perdido.

4.5 Registo persistente (log.txt)

Quando um comando termina, o controller calcula a duração desde a receção do pedido até à notificação de conclusão (usando gettimeofday()) e acrescenta uma linha ao ficheiro log.txt com: user_id, PID do runner, comando executado e duração em segundos com precisão de milissegundos.

5. Política de Escalonamento

5.1 Descrição — Round-Robin por utilizador

A política implementada é um Round-Robin por utilizador. Quando uma vaga fica disponível, o controller percorre a fila de espera (ordenada por chegada) e seleciona o primeiro pedido cujo user_id seja diferente do último utilizador autorizado. Se todos os pedidos em fila pertencerem ao mesmo utilizador, seleciona-se o mais antigo.

Esta política combina dois objetivos: fairness (alternância entre utilizadores) e respeito pela ordem de chegada dentro de cada utilizador. Em contraste com um FIFO puro, evita que um utilizador com muitos pedidos monopolize o sistema quando existem outros à espera.

5.2 Implementação

A função pick_next() implementa a seleção. A variável global last_user_id regista o utilizador do último pedido autorizado. A função fill_slots() é invocada sempre que um slot fica livre (após cada conclusão), preenchendo todas as vagas disponíveis com pedidos da fila.
O paralelismo máximo (max_parallel) é um argumento de linha de comandos do controller. Por omissão, o sistema suporta qualquer valor ≥ 1.

6. Avaliação Experimental

6.1 Metodologia

A avaliação experimental foi realizada de forma modular através de três scripts bash, garantindo que cada requisito do enunciado fosse testado isoladamente e sob carga:

- test_redirects.sh: Focado na validação dos operadores de shell. Testa a integridade dos dados em redirecionamentos (>, <, 2>) e a ausência de *deadlocks* em pipelines complexas com múltiplos estágios.
- test_scheduling.sh: Destinado a validar a lógica de escalonamento. Simula múltiplos utilizadores simultâneos para medir o impacto do paralelismo e verificar a eficácia da política de *Fairness* (Round-Robin por utilizador).
- test_parallelism.sh: Teste de *stress* intensivo que submete uma carga fixa de tarefas para medir a escalabilidade linear do sistema e o *overhead* de gestão de FIFOs do controller.

O ficheiro log.txt é limpo antes de cada cenário para garantir a integridade dos dados de auditoria.

6.2 Experimento 1 — Impacto do Paralelismo (test_scheduling.sh)

Cenário: 3 utilizadores, 4 comandos cada (12 comandos no total), com duração aleatória entre 0,2 e 0,5 s. O max_parallel varia entre 1 e 4.

Paralelo (P)	Média user 1	Média user 2	Wall-clock Total
1	1,401 s	1,657 s	4 s
2	0,700 s	1,163 s	3 s
3	0,425 s	0,765 s	1 s
4	0,427 s	0,990 s	1 s

Tabela 3 — Impacto do grau de paralelismo .

De P=1 para P=2, o *wall-clock* desce de 4 s para 3 s e os tempos médios por utilizador baixam para cerca de metade. A partir de P=3, a melhoria estabiliza pois o *bottleneck* deixa de ser o paralelismo e passa a ser a duração intrínseca dos comandos.

6.3 Experimento 2 — Fairness entre Utilizadores (test_scheduling.sh)

Cenário: 4 utilizadores, 3 comandos cada, paralelo=2. Objetivo: verificar que nenhum utilizador sofre *starvation*

Utilizador	Tempo médio real	Nº comandos	Observação
1	0,334 s	3	Chega primeiro ao FIFO
2	0,524 s	3	Round-Robin alterna com user 1
3	1,049 s	3	Aguarda libertação de slots

4	1,178 s	3	Último a ser servido, sem *starvation*

Tabela 4 — Tempos médios reais por utilizador com política Round-Robin.

Com a política Round-Robin, o controller alterna entre utilizadores a cada vaga disponível. Após autorizar um comando do Utilizador 1, o sistema seleciona prioritariamente o pedido do Utilizador 2, garantindo progressão para todos os utilizadores em fila.

6.4 Experimento 3 — Consulta durante Execução (test_scheduling.sh)

Cenário: 3 runners com sleep 3 lançados em simultâneo, paralelo=2. O runner de consulta (-c) é invocado 0,5 s após o arranque.

Output real obtido:

=== ESTADO DO CONTROLLER ===

Paralelo max : 2

A correr : 2

Em fila : 1

-- Em execucao --

PID 929 | user 1 | cmd: sleep 3

PID 930 | user 2 | cmd: sleep 3

-- Em espera (ordem de escalonamento) --

[1] PID 931 | user 3 | cmd: sleep 3

A consulta foi processada sem atrasar os runners em execução, provando que o controller serve pedidos de estado (tipo 2) de forma não-bloqueante entre ciclos de leitura

6.5 Experimento 4 — Redirecionamentos e Pipelines (test_redirects.sh)

Todos os cinco testes de redirecionamento passaram com sucesso na execução real:

Teste	Comando executado	Op.	Resultado real
1	echo ola mundo > tmp/out1.txt	>	PASSOU — conteúdo correto
2	wc -l < tmp/input.txt > tmp/out2.txt	< >	PASSOU — contagem: 3 linhas
3	ls bin | wc -l > tmp/out3.txt	|	PASSOU — 2 ficheiros contados
4	cat frutas.txt | sort | uniq > tmp/out4.txt	| | >	PASSOU — 3 frutas únicas ordenadas
5	ls /caminho_inexistente 2> tmp/out5.txt	2>	PASSOU — stderr capturado

Tabela 5 — Resultados reais dos testes de redirecionamento.

A ausência de *deadlocks* em pipelines multi-estágio (Teste 4) confirma que todos os descritores de ficheiro são corretamente fechados antes do execvp(). Cada processo filho fecha o extremo do pipe que não utiliza, garantindo que o read() do estágio seguinte recebe EOF quando o processo anterior termina.

7. Decisões de Design e Limitações

7.1 O_RDWR no controller_pipe

O controller abre o seu FIFO em O_RDWR em vez de O_RDONLY. Isto impede que o read() retorne EOF (0) quando não há runners ativos, evitando que o controller termine prematuramente enquanto aguarda novos pedidos.

7.2 Identificador único por comando

O PID do processo runner é usado como identificador único de cada comando. Como cada invocação de runner cria um processo com PID distinto atribuído pelo kernel, não é necessário um contador global, simplificando o design.

7.3 Atomicidade das escritas

A estrutura Msg tem 320 bytes, inferior ao limite POSIX de PIPE_BUF (≥512 bytes). Isto garante que escritas concorrentes de múltiplos runners no controller_pipe são atómicas, não havendo risco de mensagens entrelaçadas.

7.4 Limitações conhecidas

• O controller é single-threaded: processa um pedido de cada vez. Em cenários com muitos runners a terminar em simultâneo, pode haver pequenos atrasos na autorização de novos comandos.

• O tamanho máximo da fila (MAX_QUEUE=200) é estático. Em cenários de carga muito elevada, pedidos além deste limite são rejeitados.

• O campo command está limitado a 300 caracteres, suficiente para a maioria dos casos mas potencialmente restritivo para comandos com muitos argumentos.

• A política Round-Robin não considera a duração estimada dos comandos (não há mecanismo de prioridade por tempo de execução

8. Conclusão

O sistema implementado cumpre todos os requisitos do enunciado: execução controlada de comandos com suporte a pipelines e redirecionamentos, paralelismo configurável, política de escalonamento justa, consulta de estado em tempo real, shutdown controlado e registo persistente em log. A comunicação é feita exclusivamente via FIFOs, respeitando a restrição de não usar printf, perror ou system().

Os testes de escalabilidade confirmam que o wall-clock decresce com o aumento do paralelismo: de 4 s (P=1) para 1 s (P=3), com saturação a partir do ponto em que P igualao número de utilizadores ativos. O teste de fairness demonstrou que a política Round-Robin serve todos os utilizadores de forma progressiva: no cenário com 4 utilizadores e paralelo=2, o user mais tardio esperou 1,178 s em média, valor claramente inferior ao que resultaria de uma política FIFO pura. Os testes de redirecionamento confirmaram o correto funcionamento de todos os operadores implementados diretamente em C, sem recurso ao interpretador de comandos.

Como trabalho futuro, seria interessante implementar uma fila de prioridade baseada no tempo de espera acumulado de cada utilizador (aging), bem como um mecanismo de timeout para comandos que excedam um limite de tempo configurável.